

현대 이지웰 최종 프로젝트

티케팅 마스터

“트래픽 분산과 동시성 제어를 중심으로”

1조 · 중간 발표

2026

목차

01

문제 정의

티케팅의 본질적 문제

02

핵심 컨셉

두 단계 방어 구조

03

기술 의사결정

선택과 트레이드오프

핵심

04

다음 단계

부하 테스트 + 배포

05

데모

실제 동작 시연

서버는 왜 터지는가?

트래픽 폭주

특정 시각 동시 접속 폭주



DB 병목

한정된 DB Pool



동시성

동일 좌석 동시 예약



재고 정합성

결제 수 = 좌석 수



외부 API

외부 API 쪽의 장애



장애 전파

특정 장애가 전체 장애로 확산



서버는 왜 터지는가?

트래픽 폭주

특정 시각 동시 접속 폭주



DB 병목

한정된 DB Pool



동시성

동일 좌석 동시 예약



재고 정합성

결제 수 = 좌석 수



외부 API

외부 API 쪽의 장애



장애 전파

특정 장애가 전체 장애로 확산



두 단계 방어 구조

트래픽 폭주



동시성 제어



1단계

동시 입장 인원 제한

1만 명 → 200명 단위 분산



2단계

좌석 단위 충돌 방지

낙관적 락 + 재시도 로직

전체 플로우 (사용자 흐름)



시스템 처리 흐름

Redis ZSET 순번

200명 ALLOWED

Seat 조회

@Version 점유

PG API 호출

Booking CONFIRMED

대기열 — 문제와 선택

문 제

1만 명 동시 접속

- DB 커넥션 즉시 고갈
- 좌석 점유 단계 충돌 폭증
- 낙관적 락 재시도 누적 → p99 응답 지연



동시 부하
감소

선 택

Redis Sorted Set

- ZADD score=진입시각 → 자동 정렬
- 200명 단위 ALLOWED 처리
- UUID 입장 토큰 발급 (Queue-Token)
- 토큰 없이 좌석 API 접근 불가

대기열 — 트레이드오프

Redis Sorted Set vs Kafka

기준	Kafka	Redis ZSET ✓
회차별 큐 분리	복잡 (Topic 폭증)	key 단위 자연 분리
순번 조회	어려움	ZRANK O(logN)
운영 부담	Zookeeper 필요	단일 인스턴스

ZSET vs LIST

기준	LIST	ZSET ✓
내 순번 확인	LRANGE 전체	ZRANK 직접 조회
중복 진입 방지	별도 SET 필요	member 자체 unique
정렬 비용	입력 순서만	score 자동 정렬

폴링 부하 분석

1만 명 × 3초 폴링 ≈ 3,300 RPS — 어디에 부하가 가는가

대기열 상태 조회는 **Redis ZRANK만 호출** → DB 부하 0
Redis는 ZRANK 단순 연산으로 수만 RPS 처리 가능 → 3,300 RPS는 여유
Tomcat 스레드 풀 영향은 부하 테스트에서 검증 예정

좌석 점유 — 동시성 시나리오

🔍 두 사용자가 같은 좌석을 동시에 클릭하면?

	사용자 A	DB (Seat 테이블)	사용자 B
T1	SELECT	version = 5	SELECT
T2	UPDATE WHERE version=5 → 성공	version → 6	UPDATE WHERE version=5 → 0 row
T3	✅ RESERVED 점유		❌ OptimisticLockException
T4		status=RESERVED	재시도 → status 체크 → SEAT_ALREADY_RESERVED 응답

@Version의 진짜 역할: COMMIT 시점의 race condition을 잡음 (SELECT 시점 status 체크와는 다른 시점)

좌석 점유 — 낙관적 vs 비관적 락

기준	비관적 락 (SELECT FOR UPDATE)	낙관적 락 (@Version) ✓
DB 자원 점유	트랜잭션 동안 row lock 유지	락 자원 안 씀
처리량	낮음 (대기 발생)	높음 (병렬 진행)
충돌 시 비용	락 대기 시간	재시도 (50~100ms × 1~2회)
우리 환경 적합도	낮음 (충돌이 적는데 락 비용)	높음 (1단계로 충돌 빈도 ↓)

⚠ 낙관적 락의 한계

- 인기 좌석 동시 요청 폭증 시 재시도 누적
→ p99 지연 증가
- 충돌 좌석 식별의 한계 (어떤 좌석이 막혔는지 정확히 못 줌)

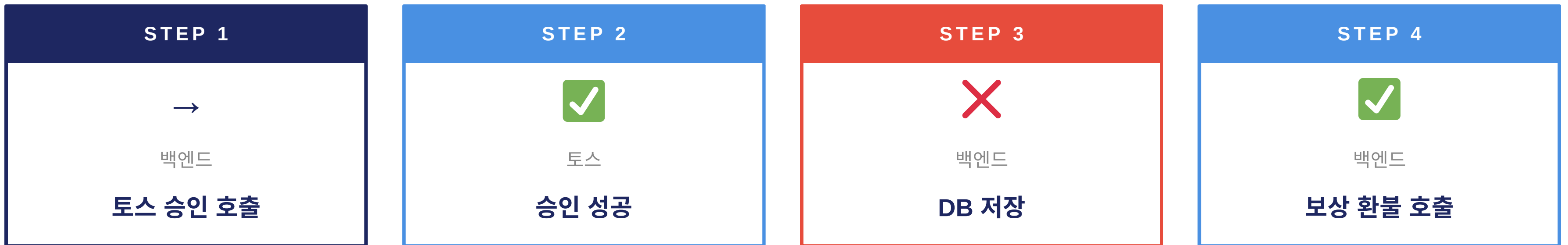
📊 다음 단계 (D3)

- 부하 테스트로 임계점 측정
- 필요 시 비관적 락 / Redis 분산 락으로 전환 검토

결제 — 보상 트랜잭션

? 토스 승인은 성공했는데, DB 저장이 실패하면?

→ 사용자 카드에서 돈은 빠졌는데, 우리 시스템엔 예매 기록 없음. 결제 도메인 최악의 시나리오



`compensateRefund(paymentKey) → tossClient.cancel(...)` 자동 호출

이중 실패 (보상도 실패) 대응

ERROR 레벨 로그 + Payment FAILED 기록 (REQUIRES_NEW 트랜잭션으로 별도 저장) → 운영자 수동 처리

결제 — 역등성 설계

같은 paymentKey가 N번 도달해도, 결제는 정확히 한 번

발생 가능 시나리오



네트워크 재시도



더블 클릭



브라우저 탭 두 개



악의적 재호출

3중 방어 (각각 다른 계층)

1차

코드 분기

findByPaymentKey() — 있으면 기존 결과 반환

2차

DB UNIQUE

Payment.payment_key UNIQUE 제약

3차

보상 환불

토스 cancel API 자동 호출

좌석 만료 + 취소

좌석 상태 머신



복구 경로 (2가지)

RESERVED → AVAILABLE: 본인 해제 / 스케줄러 만료 (30초 주기)
SOLD → AVAILABLE: 취소 시 직접 복구 (RESERVED 경유 없음)

취소 수수료 (경기 시작 기준)

남은 시간	수수료
72시간 이상	0% (전액 환불)
24~72시간	10%
24시간 미만	취소 불가

만료 설정

TTL 7분: 결제 페이지 평균 소요 시간 + 여유
스케줄러 30초 주기: reserved_until 지난 RESERVED 자동 복구

- @Version으로 만료 ↔ 점유 race 차단

환불 처리

- 백엔드가 토스 환불 API 직접 호출
- 테스트 환경에서도 동일 동작 (실제 돈 X)
- cancelReason 필드에 사용자 입력 사유 기록

부하 테스트 계획

①

대기열 처리량

1만 명이 0.5초 내 줄 설 수 있는가

②

좌석 점유 처리량

OptimisticLockException 발생률

③

결제 p99 응답 시간

토스 API 포함 end-to-end

④

DB 커넥션 풀 사용률

피크 시간대 풀 고갈 여부

도구: k6 시나리오: ① 정상 부하 ② 인기 좌석 집중 ③ 결제 실패율 의도적 증가

측정 → 결정 → 개선

낙관적 락 충돌률 측정

p99 응답 시간 기준선 대비

✓ 임계 이하

낙관적 락 유지

- 재시도 정책 미세 조정
- 모니터링 강화 (Prometheus + Grafana)
- 배포 진행

! 임계 이상

락 전략 전환 검토

- 인기 좌석만 비관적 락 (SELECT FOR UPDATE)
- Redis 분산 락 (Redisson)
- 좌석 단위 락 이원화

데모 시연

LIVE DEMO

핵심 메시지



단일 기술이 아닌, 트래픽 특성에 맞춘 단계별 방어로
트래픽 분산과 동시성 문제를 풀었다



01

두 단계 방어

대기열로 분산 → 낙관적 락으로 충돌 제어

02

엣지 케이스 설계

보상 트랜잭션 · 먹등성 · 만료 스케줄러

03

측정 기반 진화

부하 테스트 → 한계 측정 → 다음 단계 결정

Q & A

질문 주세요

감사합니다

현대 이지 웰 최종 프로젝트 1조